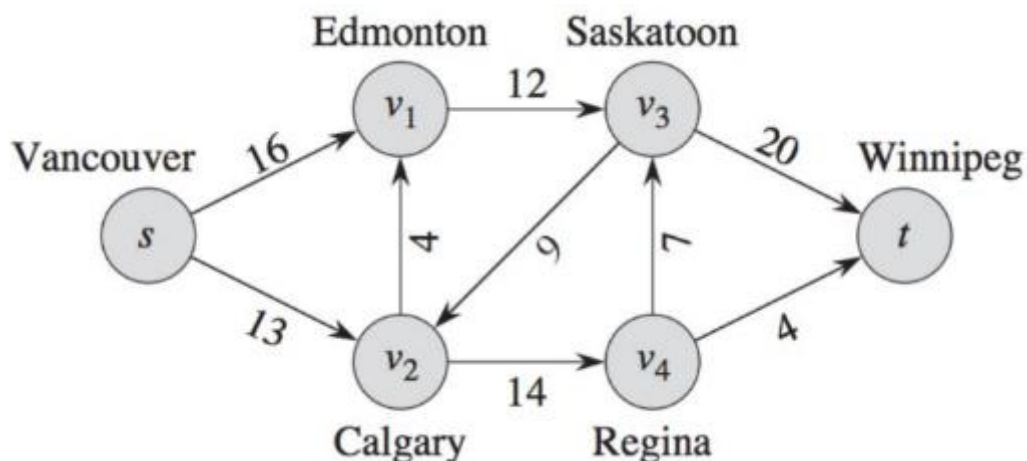


Backtracking y SDL | Actividad:

La actividad tiene como objetivo comprender y aplicar una "propuesta" para resolver un problema histórico como lo es el Problema de los Puentes de Königsberg que pregunta si es posible dar un paseo por la ciudad cruzando cada uno de los siete puentes exactamente una vez y volviendo al punto de inicio. Esto con la finalidad de entender cómo funciona el Backtracking y los Árboles SDL en la programación lógica (Específicamente en PROLOG)

Para ello, se propone una versión más simple del problema mediante grafos dirigidos:



A partir de esta estructura se plantean unas preguntas (Queries) que nos permitirán construir las reglas y hechos en PROLOG:

- ¿Existe una **conexión** entre Saskatoon y Vancouver?
- ¿Con qué nodos está **conectado** Regina y cuál es el costo de cada conexión?
- ¿Un nodo X tiene aristas?
- ¿Cuál es el costo para ir de un nodo X a un Z pasando por Y?
- ¿Es posible **viajar** desde Edmonton a Calgary?

De acuerdo con estas reglas construiremos la base de conocimiento de nuestro programa junto con los hechos y reglas que harán que podamos resolver estos problemas. Cada hecho define una relación directa entre dos ciudades junto con un costo asociado, bajo la palabra clave **conexión** que es mencionada en los problemas descritos anteriormente. Este tercer parámetro (costo) actúa como una magnitud o métrica, permitiendo representar no solo "tarifas" de traslado, sino cualquier otra referencia cuantitativa (por ejemplo, distancia o tiempo) entre un punto A y un punto B.

%Hechos

conexion(vancouver, edmonton, 16).
conexion(vancouver, calgary, 13).
conexion(edmonton, saskatoon, 12).
conexion(saskatoon, winnipeg, 20).
conexion(calgary, edmonton, 4).
conexion(saskatoon, calgary, 9).
conexion(calgary, regina, 14).
conexion(regina, saskatoon, 7).
conexion(regina, winnipeg, 4).

%Reglas

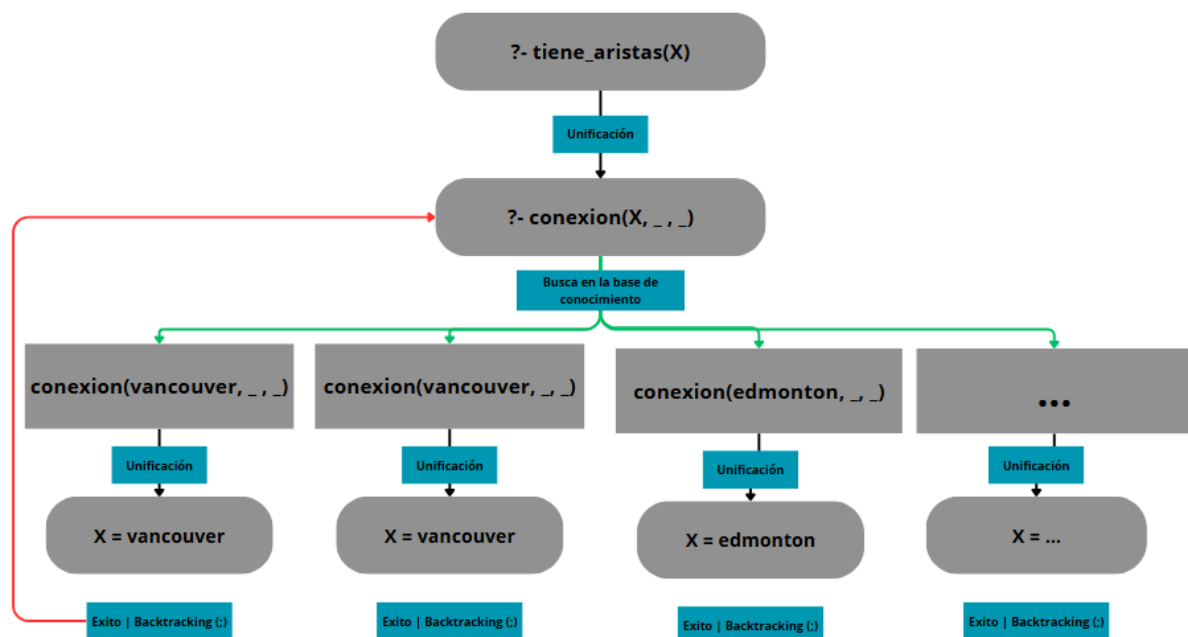
- La regla **tiene_aristas(X)** determina si un nodo X posee al menos una conexión saliente, independientemente de cuál sea su destino y su costo asociado. Para lograr esto, se emplean variables anónimas (representadas con el guion bajo _), las cuales se utilizan cuando los valores específicos de dichos argumentos no son necesarios para la evaluación de la regla.

tiene_aristas(X) :-

conexion(X, _, _).

Para comprender como funciona analizaremos esta regla mediante la consulta:

?- tiene_aristas(X)



En primer lugar, Prolog establece la consulta **?-tiene_aristas(X).** e intenta unificarla con la base de conocimientos. Al encontrar la regla **tiene_aristas(X) :- conexion(X, _, _).**, la sustituye por **?-conexion(X, _, _).**, la cual indica que se debe encontrar una variable X que sea el punto de origen de alguna conexión registrada.

A partir de este punto, se recorre los hechos de **conexion** en el Árbol SLD de arriba hacia abajo, aplicando backtracking cada vez que se le solicita una nueva respuesta:

- **Primer éxito:** Se unifica con el primer hecho **conexion(vancouver, edmonton, 16).** La variable se asigna como **X = vancouver** y la consulta devuelve éxito.
- **Primer backtracking:** Al pulsar ";", Prolog retrocede para buscar otra alternativa y unifica con el segundo hecho **conexion(vancouver, calgary, 13),** devolviendo nuevamente **X = vancouver.**

Este proceso de backtracking continúa secuencialmente explorando cada hecho restante (calgary, saskatoon, calgary, regina, regina) hasta agotar los hechos de la base de datos. Una vez evaluado el último hecho **conexion(regina, winnipeg, 4)** y al no existir más alternativas en el Árbol SLD, el backtracking finaliza retornando false.

- La regla **costo(X, Z, Y, COSTO)** calcula el costo acumulado de una ruta que parte de un nodo origen X y llega a un destino Z, atravesando un único nodo intermedio Y (lo que representa un camino de longitud fija de dos saltos). El funcionamiento se basa en consultar dos conexiones consecutivas para extraer sus respectivos costos (COSTO1 y COSTO2), utilizando posteriormente el operador aritmético **"is"** para evaluar la suma y asignar el resultado al argumento COSTO. Cabe destacar que esta regla está acotada exclusivamente a trayectos con un solo nodo intermedio y no maneja la recursividad para rutas de mayor complejidad.

costo(X, Z, Y, COSTO) :-

**conexion(X, Y, COSTO1),
 conexion(Y, Z, COSTO2),
 COSTO is COSTO1+COSTO2.**

- A diferencia de las reglas anteriores, la regla **viajar(X, Y)** permite determinar el alcance entre dos nodos X e Y sin importar cuántos nodos intermedios existan, gracias al uso combinado de recursividad y backtracking.

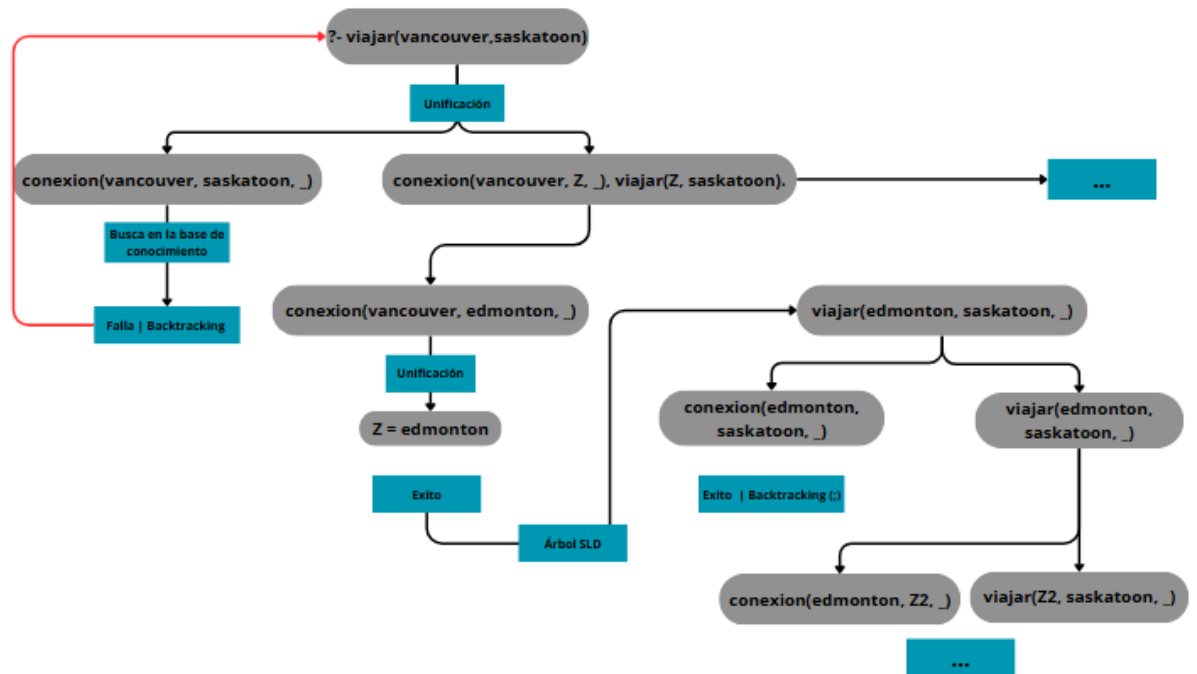
Esta lógica se divide obligatoriamente en dos cláusulas independientes (que actúan como una disyunción lógica o 'OR'):

- **El caso base:** Verifica si existe una conexión directa.
- **El caso recursivo:** Avanza a través de un nodo intermedio Z y se llama a sí mismo para continuar la ruta.

Definirlas por separado es indispensable. Si se intentara unificar ambas condiciones en una sola regla mediante una conjunción (operador ","), se exigiría que toda búsqueda cumpla simultáneamente con una conexión directa y un tramo recursivo, lo cual invalidaría toda la regla para trayectos largos.

```
viajar(X, Y) :-
    conexion(X, Y, _).
viajar(X, Y) :-
    conexion(X, Z, _),
    viajar(Z, Y).
```

Para evaluar la regla de **viajar(X,Y)** mediante la consulta **?- viajar(vancouver, saskatoon).**, Prolog combina el Árbol de Resolución SLD con backtracking a través de sus dos cláusulas (caso base y caso recursivo):



En primer lugar, Prolog intenta resolver la meta mediante la regla del caso base **viajar(X,Y) :- conexion(X,Y,_)**. Al buscar el hecho **conexion(vancouver, saskatoon, _)**, comprueba que no existe una conexión directa, resultando en un fallo para esa rama del Árbol SLD.

Inmediatamente, se realiza backtracking a la meta inicial y evalúa el caso recursivo **viajar(X,Y) :- conexion(X,Z,_), viajar(Z,Y)**. Esto transforma la consulta en la búsqueda de una submeta: **conexion(vancouver, Z, _)** seguida de **viajar(Z, saskatoon)**. Prolog encuentra la primera coincidencia en la base de hechos con **conexion(vancouver, edmonton, 16)**, unificando la variable **Z = edmonton**. A continuación, genera una llamada recursiva para resolver **viajar(edmonton, saskatoon)**. En esta nueva rama, vuelve a evaluar el caso base buscando **conexion(edmonton, saskatoon, _)**, la cual coincide con el hecho **conexion(edmonton, saskatoon, 12)**, alcanzando un nodo de éxito (true).

Si se fuerza backtracking solicitando más soluciones con ";", Prolog retrocede sobre la rama de resolución. Deshace las unificaciones previas y explora las alternativas de **conexion(vancouver, Z, _)**, tomando como siguiente opción **conexion(vancouver, calgary, 13)** con **Z = calgary**. A partir de ahí, explora recursivamente caminos alternativos (como Vancouver -> Calgary -> Edmonton -> Saskatoon o Vancouver -> Calgary -> Regina -> Saskatoon), ramificando nuevamente el Árbol SLD hasta agotar todas las combinaciones posibles

- La regla **viajarCosto(X, Y, COSTO)** extiende la lógica de alcance del predicado **viajar(X, Y)** al añadir la suma de costos de las aristas del grafo. Mientras que el primero solo retorna un valor de verdad (verdadero o falso), esta regla calcula y devuelve el costo total de la ruta. Su funcionamiento opera de forma recursiva:

- **Caso base:** Retorna directamente el costo de una conexión directa.
- **Caso recursivo:** Desglosa el trayecto en un tramo inicial (COSTO1) y el resto del camino (COSTO2), sumándolos mediante el operador aritmético is.

Por ejemplo, al consultar **viajarCosto(vancouver, winnipeg, COSTO)**, el sistema evalúa la ruta compuesta vancouver -> edmonton -> saskatoon -> winnipeg, acumulando paso a paso los costos individuales (16 + 12 + 20) hasta unificar y retornar el costo total de 48.

viajarCosto(X,Y,COSTO) :-

conexion(X,Y,COSTO).

viajarCosto(X,Y,COSTO) :-

conexion(X,Z,COSTO1),

viajarCosto(Z,Y,COSTO2),

COSTO is COSTO1 + COSTO2.